

Design and Implementation of a Transport & Delivery Management Information System

Academic Context

Degree: 3rd Year License (L3) – Computer Science (Informatique)

Module: Information Systems 2 (SI2)

University: USTHB – University of Science and Technology Houari Boumediene

Team N°: E23

Members:

- Mati Ahmed Maher 232331423816 Group4
- Mazouzi Abdelkader 232331425818 Group4
- Djabali Adem 232331566013 Groupe4
- Sid Aimen 232331609003 Groupe4

Abstract

This report presents the design and development of a Transport and Delivery Management Information System implemented using the **Django web framework**. The system was developed as part of the Information Systems 2 module in the 3rd year License in Computer Science at USTHB. The objective is to model, implement, and validate a real-world information system that manages transport and delivery operations, including clients, shipments, tours, billing, payments, incidents, and dashboards.

The project follows a structured information system methodology, starting from requirements analysis and data modeling (MCD/MLD) to implementation and testing. Particular emphasis is placed on the **Django backend architecture**, ORM usage, security mechanisms, and business logic implementation.

1. Introduction

In the transport and delivery sector, companies must manage a large volume of operational data such as shipments, clients, drivers, vehicles, invoices, and payments. Manual management or poorly structured systems often lead to errors, delays, and loss of data integrity.

This project aims to design and implement a centralized Information System that automates and secures these operations. The system is implemented as a **web application using Django**, which provides a robust framework for building secure, scalable, and maintainable information systems.

The report redevelops the academic project into a realistic application while strictly respecting the educational objectives of the **L3 Informatique – USTHB** curriculum.

2. Project Objectives

The main objectives of this project are: - Apply information system concepts studied in SI2 - Design a complete data model using Merise and UML - Implement business logic using Django - Ensure data integrity using relational databases - Develop a secure, role-based web application - Produce a clear and structured academic report

3. Requirements Analysis

3.1 Functional Requirements

3.1.1 Client Management

- Create, update, delete, and research clients
- Store personal and contact information
- Track client financial balance

3.1.2 Shipment Management

- Register shipments with weight, volume, and description
- Automatically generate unique tracking numbers
- Associate shipments with clients, destinations, service types, drivers, and vehicles
- Manage shipment status lifecycle

3.1.3 Package Management

- Create, update, delete, research packages
- Associate packages with shipments

3.1.4 Invoices and Payments

- Automatically generate invoices per client
- Calculate VAT at 19%
- Manage partial and full payments
- Update client balance automatically

3.1.5 Incident and Reclaims Management

- Register incidents related to shipments or tours
- Track incident status and resolution
- Manage customer claims

3.1.6 Dashboards and Analytics Management

- Display shipment statistics
- Show revenue and invoice summaries
- Present operational indicators

3.1.7 Drivers and Vehicles

- Create, update, delete, research Drivers and Vehicles
- Assign drivers to vehicles

4. Non-Functional Requirements

- **Security:** Authentication and authorization of agents
 - **Performance:** Optimized database queries
 - **Usability:** Simple and intuitive interface
 - **Scalability:** Modular Django applications
 - **Maintainability:** Clean code and documentation
-

5. Relation Between Models

5.1 Main Entities

- Client
- Shipment
- ServiceType
- Destination
- Driver
- Vehicle

- Invoice
- Payment
- Incident
- Reclaim
- Package
- Agent
- AgentToken

5.2 Important Business Rules

- A client can have multiple shipments
- A shipment belongs to one client only
- A shipment is assigned to one driver, one vehicle, one destination and one service type
- An invoice groups shipments of a single client
- A payment is linked to one invoice
- A reclaim is assigned to one agent and one or multipole packages and invoices
- An agent has one unique agent token

5.3 Explanation of Data Relationships and Constraints

- The data model of the system is centered around the Client and Shipment entities. A client can create multiple shipments, but each shipment is associated with exactly one client. This one-to-many relationship ensures traceability and accountability.
- Shipments are linked to ServiceType, Destination, Driver, and Vehicle entities through foreign keys. This design allows each shipment to be precisely described in terms of service conditions, delivery location, and assigned resources.
- Invoices are generated per client and may group multiple shipments. Payments are linked to a single invoice, ensuring clear financial tracking. Many-to-many relationships, such as those between Reclaims and Packages or Invoices, are implemented using associative tables to maintain database normalization.
- Business constraints such as preventing the deletion of paid invoices or shipment already assigned to a driver are enforced at both database and application levels.

6. Model Attributes

CLIENT (id, last name, first name, email, phone, address, due balance, created at)

DRIVER (id, first name, last name, email, phone number, address, license number, availability, hire date, active, created at)

AGENT (id, first name, last name, email, username, password hash, account status, created at)

VEHICLE (id, registration number, type, brand, model, year, license plate, capacity, status, fuel mileage, start date, end date, active, created at)

DESTINATION (id, city, state, country, base fare, created at)

SERVICETYPE (id, name, description, volume surcharge, weight surcharge, priority level, created at)

INVOICE (id, number, issue date, subtotal, tax amount, total amount, status, created at)

SHIPMENT (id, tracking number, total weight, total volume, status, expected delivery date, delivery date, notes, created at)

PACKAGE (id, weight, volume, description, value, created at)

PAYMENT (id, number, date, amount, method, status)

INCIDENT (id, type, severity, description, location, date, is resolved, notes)

RECLAIM (id, number, type, description, date, resolution date, reclaim status, priority, created at)

AGENT_TOKEN (id, token, created at)

Remark:

- There are models of many to many relations:

RECLAIM_PACKAGES (id, created at)

RECLAIM_INVOICE (id, created at)

- There is a model of the confirmation code send it to agent:

PASSWORD_RESET_CODE (id, code, created at, is used)

- The MCD and the MLD are the Conception document with the application.

7. Django-Based System Architecture

7.1 Global Architecture

The system follows a **three-tier architecture**:

- ❖ Presentation Layer: HTML templates rendered by Django and designed with CSS
- ❖ Application Layer: Django views and business logic
- ❖ Data Layer: Relational database managed via Django ORM

8. Django Implementation Details (Core Part)

8.1 Project Structure

The Django project is organized into multiple applications:

- accounts: user authentication and roles
- clients: client management
- shipments: shipment and tracking logic
- billing: invoices and payments
- incidents: incident and claims management
- dashboard: statistics and analytics

This modular structure improves maintainability and scalability.

8.2 Django Models

Each entity in the MCD is implemented as a Django model.

Example: Shipment Model

- tracking_number: CharField (unique)
- weight: FloatField
- volume: FloatField
- status: CharField (choices)
- client: ForeignKey(Client)
- destination: ForeignKey(Destination)
- service_type: ForeignKey(ServiceType)

Django ORM automatically ensures referential integrity.

8.3 Business Logic in Django

Business rules are implemented at different levels:

- Model methods (e.g. price calculation)
- Django signals (automatic invoice updates)
- View-level validations

Example: - Prevent deletion of invoice assigned to client - Automatic due balance calculation before delete an invoice

8.4 Views and URL Routing

The application uses Django views to handle HTTP requests.

- Function-based views for simple CRUD
- Class-based views for complex operations

URLs are mapped clearly to each module, following REST principles.

8.5 Forms and Validation

Django Forms are used to: - Validate user input - Protect against invalid data - Improve user experience

Custom validation rules are implemented for business constraints.

8.6 Authentication and Security

- Django authentication system
 - Role-based access control
 - Login-required decorators
 - Protection against CSRF attacks
-

8.7 Django ORM and Database Migrations

- Django ORM plays a central role in the implementation of the system. It provides an abstraction layer between Python objects and relational database tables, allowing developers to manipulate data without writing raw SQL queries.
- Each Django model corresponds to a database table, while fields represent table columns. Relationships such as One-to-Many and Many-to-Many are defined using ForeignKey and ManyToManyField, ensuring referential integrity.
- Database migrations are used to version-control the database schema. Every change to a model generates a migration file, which can be applied safely to update the database structure.

9. Pricing and Billing Logic

Shipment price calculation:

Total HT = Base Fare + (Weight × Price per Kg) + (Volume × Price per m³)

VAT calculation:

Total TTC = Total HT + (Total HT × 0.19)

All calculations are automated using Django models and services.

10. Testing and Validation

- Unit tests for pricing logic
- Functional tests for CRUD operations
- Manual testing with realistic scenarios

Testing ensures system reliability and correctness.

11. Deployment

The application can be deployed using: - Django development server (academic context) - PostgreSQL or MySQL database

The modular design allows easy migration to production.

12. Difficulties and Solutions

- **Complex relationships:** solved using Django ORM
 - **Business constraints:** enforced via model validation
 - **Performance:** optimized queries using select_related
-

13. User Manual

- The system is designed to be intuitive and easy to use. Users authenticate using a secure login interface. After authentication, agents can access the dashboard according to their assigned roles.
- To create a client, the agent navigates to the client management section and fills in the required information. Shipments can then be registered by associating them with an existing client and selecting the appropriate service type and destination.

- Invoices are generated automatically based on completed shipments. Payments can be registered partially or fully, and client balances are updated automatically.

14. Conclusion

This project demonstrates the practical application of information system concepts using Django. It successfully meets the academic objectives of the L3 Informatique program at USTHB and provides a realistic, functional transport and delivery management system.

The project bridges theory and practice by combining Merise modeling, relational databases, and modern web development using Django.

15. Project Organization and Teamwork

- The project was carried out as a team effort, with tasks distributed among members according to their skills. Collaboration was ensured through regular meetings and task coordination.
- This collaborative work allowed the team to improve both technical and organizational skills, while respecting academic deadlines and quality requirements.

16. Future Improvements

- REST API with Django REST Framework
- Mobile application integration
- GPS tracking
- Advanced analytics